

Simulator — virtual feeder

Owner: *TBD*

Depends on: `docs/PROTOCOL.md`

Deliverable for M1: protocol-conformant telemetry indistinguishable from real nodes, plus injectable fault scenarios.

You are the critical path for two other people. Backend and frontend cannot start until this emits data, so ship a crude version on day one and refine after.

1. What this is for

Two jobs, and the second is the one people forget:

1. **Unblock the demo.** Feed the backend realistic telemetry so the dashboard has something to show.
2. **Test the arbiter against cases you cannot produce on a bench.** You will never physically create a substation outage, a lightning transient and a monsoon burst on demand. The simulator is how `decide()` gets validated against the scenarios that would otherwise only be arguments in a slide.

2. Feeder model

Use **real coordinates**. Pull an actual LT distribution line from OpenStreetMap in the Thiruvananthapuram area, take 12 pole positions along it, and store them in `feeders/kseb_tvm_f12.json`. Judges notice invented geometry, and a real polyline costs you thirty minutes.

```
{
  "feeder_id": "KSEB-TVM-F12",
  "substation": { "lat": 8.5241, "lng":
76.9366 },
  "nodes": [
    { "node_id": "N-001", "lat": 8.5245,
"lng": 76.9371, "span_m": 42 },
    { "node_id": "N-002", "lat": 8.5249,
"lng": 76.9378, "span_m": 45 }
  ]
}
```

3. Signal model

Each node emits at 2 Hz:

```
efield_rms = nominal
              × diurnal(t)           slow ±4%
over 24 h, load-driven
              × weather(t)           scenario-
driven multiplier
              × (1 + noise)          Gaussian,  $\sigma$ 
= 0.03
```

```
× break_mask(node)    0.02 if  
downstream of an active break
```

Get the noise right. A perfectly clean signal makes the detector look trivial and invites the question "what happens in rain?" – which you then answer with a button press.

`baseline` is an EWMA with $\tau \approx 10$ min, computed by the simulator exactly as the firmware will, so the demo's `deviation_pct` behaves like the real thing.

4. Scenarios

Scenario	Behaviour	Expected outcome
<code>break_mid_feeder</code>	field $\rightarrow$ ~2% for N-007 and all downstream, permanent	<code>ISOLATE</code> , span N- 006 $\leftrightarrow$ N-007
<code>break_at_tail</code>	last two nodes collapse	<code>ISOLATE</code> , span at tail
<code>rain_burst</code>	all nodes -25% for 8 s, then recover	no trip; nodes touch <code>SUSPECT</code> and return

Scenario	Behaviour	Expected outcome
vegetation_contact	one node -45%, fluctuating, sustained	ALERT only, no isolate
substation_outage	every node → ~1% simultaneously	no trip — global collapse veto
switching_transient	300 ms spike then normal	no trip — fails sustain
node_offline	one node stops transmitting	ALERT , node greys out
low_battery	one node's battery_mv drifts to 3100	maintenance alert
reset	all nodes back to NORMAL	clean slate for the next demo run

Each scenario is a coroutine that mutates simulator state over time and then exits. They must be composable — the demo where you fire `rain_burst`

during a `break_mid_feeder` and the system still gets the span right is a strong moment.

5. Structure

```
simulator/  
├─ feeders/kseb_tvm_f12.json  
├─ sim/  
│   ├─ node.py          per-node signal  
│   │   generation, baseline EWMA  
│   └─ feeder.py        feeder  
│   │   orchestration, tick loop  
│   └─ scenarios.py      one coroutine per  
│   │   scenario  
│   └─ bus.py            async queue the  
│   │   backend consumes  
└─ tests/test_scenarios.py
```

The simulator publishes to an `asyncio.Queue` that `backend/app/ingest.py` consumes. Later, the MQTT ingest path implements the same interface and the simulator is simply not started. **No backend code changes when real hardware arrives** — that is the whole point of the interface.

6. Task list

- [] Day 1: 12 nodes, 2 Hz, noise, straight into the queue. Crude but running.

- [] Day 1: `break_mid_feeder` and `reset` – enough for the backend to test `decide()`
- [] Day 2: diurnal drift, baseline EWMA matching firmware behaviour
- [] Day 2: remaining scenarios
- [] Day 3: scenario composition, `POST /simulate` wiring
- [] Day 4: export every scenario as a JSON fixture into `docs/vectors/` – this is how firmware and gateway get tested against identical data
- [] Later: replay mode – feed recorded real telemetry from bench characterisation back through the same pipeline

7. Acceptance criteria

1. Telemetry validates against `backend/app/models.py` with zero coercion.
2. Every scenario produces the expected arbiter outcome in the table above.
3. Scenarios are deterministic under a fixed random seed, so vectors are reproducible.
4. Running for 30 minutes produces no drift-induced false SUSPECT.

8. Gotcha

Resist making the break signature cleaner than reality.
If your simulated collapse is a perfect step to zero, your
thresholds get tuned to a fantasy and the first real
bench test embarrasses you. Once hardware/
produces characterisation data, come back and re-fit
the noise and collapse profile to what the actual probe
does.